Computer Architecture

ELL 782

Report

Assignment 2

By Suchet Bhalla & Shardul Yadav

2022EET2088 & 2022EET2090

Objective: To create a **C**onvolutional **N**eural **N**etwork by using myHDL.

A typical CNN has the following architecture:

[INPUT] → [CONV 1] → [ReLU] → [POOL 1] →

[CONV 2] → [ReLU] → [POOL 2] → [FC LAYER] → [RESULT]

We worked to create the layers which perform **convolution** & **pooling**.

Note: We did not implement the ReLU layer, because we limited the values to positive integers.

Overview of the stages in our circuit:

1. Padding the input-matrix
2. RAM: Transfer elements sequentially, from matrix to 9 buffers
3. Convolution

   a. Wallace Tree Multiplier (4 bit)

   b. Carry Look Ahead Adder (8 bit)

4. RAM: Transfer elements sequentially, from buffer into a new matrix
5. RAM: Transfer elements sequentially, from this matrix to 4 buffers
6. Max. pooling
7. RAM: Transfer elements sequentially, from a buffer into a new matrix

Specification of the Input:

The matrix can be of arbitrary size, but its dimensions (i.e., no. of rows & columns) must be even numbers. This constraint is imposed by the Max. Pooling layer (because its stride length is 2).

- Each element (of this matrix) must be a 3-bit integer in the range 0 to 7; The element-size was limited to 3 bits, to ensure that the Sum-of-Products (i.e., the result of convolution) has size 16 bits.

Details of our circuit:

Given a matrix & its dimension,

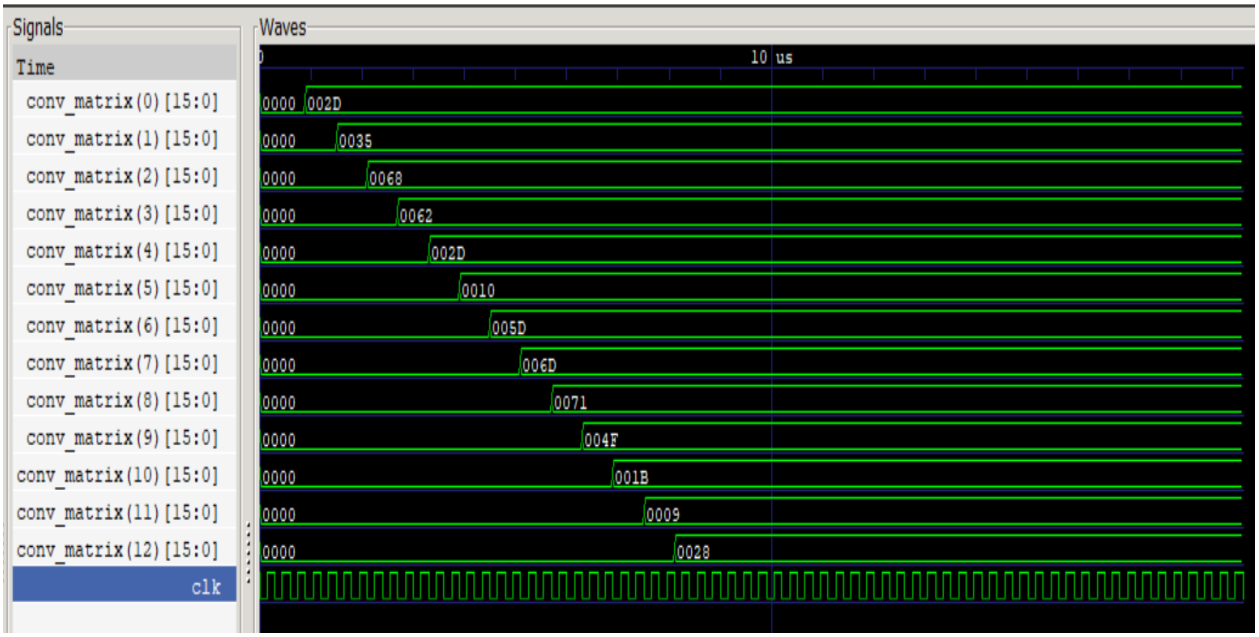
1. **Padding:** it is zero-padded by 2 columns (right) & 2 rows (bottom)

2. **RAM:** The *kernel* size is 9, so for performing 1 *convolution*, we require 9 elements from the (padded) matrix. Consequently, our program transfers 9 elements into 9 **buffers,** once in every <u>2 clock-cycles</u> **;** the cause of this inefficiency is stated in the step 4.

- These buffers are *enabled* & *reset* via asynchronous signals.
- The clock-period is 300 *nano*-seconds**;** this duration will be justified, in the next point.

3. **Convolution:** In every clock-cycle, we perform one convolution, by using a combinational circuit.

- In practice, the logic-gates (in this combinational circuit) will induce delays, typically 10 *nano sec* per gate & 1 *nano sec* per 15 cm of wire. To account for this, we select a suitable clock-period.
  a. **Multiplication:** We used 9 Wallace Tree Multipliers for multiplying the elements of the matrix (3 bit) & the kernel (4 bit).
- These multiplications are performed parallelly.
- Each WTM is capable of multiplying 2 inputs of size 4 bits; We limited the input-size to 3 bits to ensure that the result (of convolution) has size 16 bits.
  b. **Addition:** We used 8 **C**arry-**L**ook-**A**head adders (8 bit) to calculate the **S**um-**o**f-**P**roducts (16 bit).
- The configuration (of these adders) ensures maximal *parallel processing*.
- **Disclaimer:** Each 8-bit CLA is composed of two 4 bit CLAs. Thus, a carry-over is propagated via a ripple.
- The following snippet displays, some output-signals of the *convolution layer*.



- For your reference, we have included the **.vcd** along with our report.

4. **RAM:** The SoP is transferred to a new matrix (for *max. pooling*.), once in every two *clock-cycles*.

- **Justification for wasting a cycle:** Our <u>program</u> performs step 1 & 4, **under 1 *for* loop**, i.e., it transfers,
    - elements (from a buffer) into a new matrix &
    - elements from the input-matrix into 9 buffers,

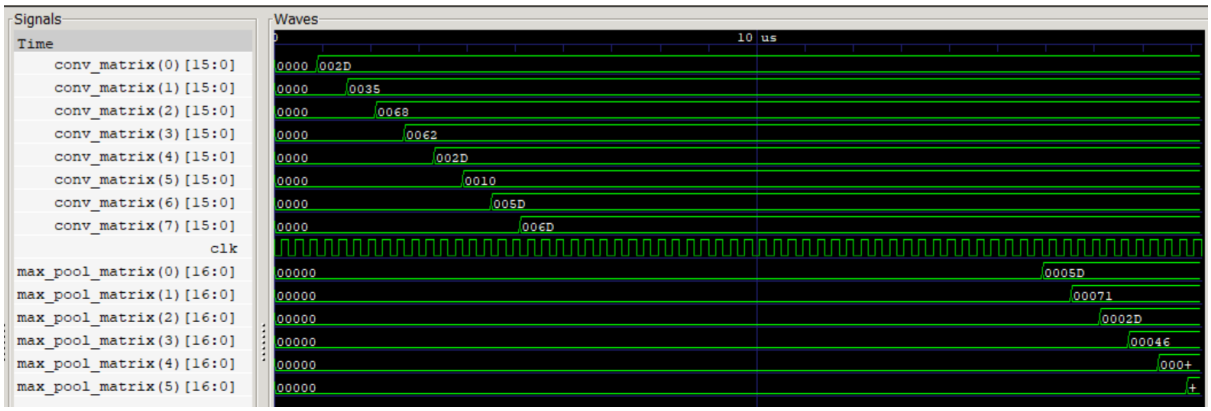
  under 1 *for* loop.

    - We were unable to segregate these steps (1 & 4), else we could have performed the transfers per cycle (as opposed to once per two cycles).

5. **RAM:** For performing *max. pooling*, we transfer 4 elements from the previous matrix into 4 buffers.

6. **Maximum Pooling:** We used 3 CLAs for this task.

- We performed subtraction of the two inputs via **2s** complement. By looking at the sign-bit of their sum, we selected the greater number.
- The following snippet displays, <u>some</u> output-signals of the *convolution layer* & *max. pooling layer*.



7. **RAM:** The max. value is transferred to a final matrix (for storage) via a buffer.

Result: We have successfully implemented a convolutional & max. pooling layer.

The following snippets are printed outputs from our myHDL program,

Input:

```
The matrix is
 7  5  1  2  4  2

 4  1  4  4  3  3

 7  5  5  2  3  6

 5  1  4  3  4  5
```

Padded-matrix:

```
The padded-matrix is
 7  5  1  2  4  2  0  0

 4  1  4  4  3  3  0  0

 7  5  5  2  3  6  0  0

 5  1  4  3  4  5  0  0

 0  0  0  0  0  0  0  0

 0  0  0  0  0  0  0  0
```

After Convolution:

```
Time elapsed  0 : Buffers have been reset
Time elapsed 300 : Buffers have been enabled
-------------------------------------------------------
Time elapsed 15000 nsec: Buffers have been disabled
------------------------------
The convolution-matrix is
 77   59   79   63   26   20

 83   76   78   78   36   18

 91   60   56   91   38    6

 50   51   59   73   29    5
```

After max. pooling:

```
Time elapsed 19200 nsec: Buffers have been disabled
-----------------------------------------------------
The max_pool_matrix is
 83   79   36

 91   91   38
```